INSTITUTO POLITÉCNICO NACIONAL

UNIDAD PROFESIONAL INTERDISCIPLINARIA DE INGENIERÍA

Y CIENCIAS SOCIALES Y ADMINISTRATIVAS

Licenciatura en Ciencias de la Informática

Reporte 2do Departamental

Abstracción y Uso de Datos (3CM30)

Alumno:

Carrera Rojo Jane Adriana

Frias Martinez Vanessa Judith

Saavedra Rodríguez Aarón Emanuel

Docente:

Entzana Garduño Yventz

02 de mayo de 2026

ÍNDICE

Introducción
SortBubble: Ordenamiento Burbuja
BubbleArr: Ordenamiento Burbuja Directo con Arreglos
BubblePtr: Ordenamiento Burbuja Indirecto con Arreglo de Índices
SortMerge: Ordenamiento por Mezcla
MergeArr: Merge Sort Directo con Arreglos
MergePtr: Merge Sort Indirecto con Arreglo de Índices
SortQuick: Ordenamiento Rapido
QuickArr: Quick Sort Directo con Arreglos
QuickPtr: Quick Sort Indirecto con Arreglo de Índices
Pilas: Estructura de Datos LIFO
Implementación con Arreglos Estáticos
Implementación con Punteros y Memoria Dinámica
Implementación con la Biblioteca Estándar STL
Colas: Estructura de Datos FIFO
Implementación con Arreglos Estáticos
Implementación con Punteros y Memoria Dinámica
Implementación con la Biblioteca Estándar STL
Listas: Estructura de Datos Lineal
Implementación con Arreglos Estáticos
Implementación con Punteros y Memoria Dinámica
Implementación con la Biblioteca Estándar STL
Conclusion

Introducción

El proyecto 2doDep reúne 24 programas escritos en C++ cuyo propósito es demostrar, de manera comparativa, la implementación de los principales algoritmos de ordenamiento y estructuras de datos lineales. Cada programa aborda el mismo problema funcional desde una perspectiva de implementación distinta, lo que permite al estudiante contrastar los enfoques y comprender sus ventajas e inconvenientes.

La organización del repositorio refleja una jerarquía de tres niveles. El primer nivel distingue el tipo de operación: ordenamiento (SortBubble, SortMerge, SortQuick) o estructura de datos (Pilas, Colas, Listas). El segundo nivel especifica la estrategia de almacenamiento empleada (Arreglos, Punteros, Librerías). El tercer nivel diferencia el modelo de dato utilizado internamente (DatoBase para tipo primitivo, Nuevo Dato para clase envoltorio).

Este reporte documenta cada programa indicando su ruta dentro del proyecto, su propósito, su lógica de operación y las decisiones de diseño que lo distinguen de sus variantes.
El orden de presentación sigue estrictamente la jerarquía del repositorio.

SortBubble: Ordenamiento Burbuja

El módulo SortBubble contiene dos programas que implementan el algoritmo de ordenamiento burbuja (Bubble Sort) aplicado a tres tipos de datos: enteros (int), caracteres (char) y objetos de la clase Persona, estos últimos ordenados por el campo edad. Ambos programas comparten la misma clase Ordenador y la misma interfaz de usuario, diferenciándose únicamente en la estrategia de intercambio.

BubbleArr: Ordenamiento Burbuja Directo con Arreglos

El programa BubbleArr implementa el algoritmo de ordenamiento burbuja intercambiando físicamente los valores dentro del arreglo. La clase Ordenador expone tres métodos (burbuja Enteros, burbuja Chars y burbuja Personas) que recorren el arreglo comparando pares de elementos adyacentes y realizando el intercambio in-place cuando el orden relativo es incorrecto. Este proceso se repite hasta que ningún intercambio es necesario, garantizando que el arreglo quede ordenado de forma ascendente.

Desde el punto de vista de complejidad, el algoritmo tiene un comportamiento cuadrático O(n^2) en el caso promedio y en el peor caso, lo que lo hace ineficiente para conjuntos de datos grandes. Sin embargo, su implementación es sencilla y didáctica, y resulta óptimo cuando el arreglo ya está casi ordenado, donde puede aproximarse a O(n). El programa emplea arreglos estáticos de tamaño fijo declarados en la pila del programa, por lo que no requiere gestión manual de memoria dinámica.

La función main solicita al usuario la cantidad de elementos y sus valores para cada tipo de dato, invoca el método de ordenamiento correspondiente y muestra el resultado. La clase Persona encapsula nombre (string) y edad (int), permitiendo ilustrar como el mismo algoritmo puede generalizarse a objetos mas complejos simplemente adaptando el criterio de comparación. Este programa sirve como referencia base frente a la cual se compara la variante indirecta BubblePtr.

BubblePtr: Ordenamiento Burbuja Indirecto con Arreglo de Índices

El programa BubblePtr implementa una variante indirecta del ordenamiento burbuja en la que los datos originales no se mueven. En lugar de intercambiar los elementos del arreglo principal, el algoritmo mantiene un arreglo auxiliar de índices enteros (int* idx) asignado dinámicamente con new. Cada posicion idx[i] almacena el indice del elemento que debe ocupar la posición y en el orden lógico; al comparar, se accede a arr[idx[j]] y arr[idx[j+1]], y si es necesario se intercambian los índices, no los datos.

Esta técnica, conocida como ordenamiento indirecto o por tabla de índices, ofrece una ventaja importante cuando los elementos del arreglo son objetos grandes o costosos de copiar: al mover solo enteros en el arreglo de índices, el costo de cada intercambio se reduce significativamente. En el caso del arreglo de objetos Persona, esto evita copiar structs completos y puede mejorar el rendimiento en la práctica, aunque la complejidad asintótica sigue siendo O(n^2).

El programa libera la memoria de los tres arreglos de índices al finalizar mediante delete[], siguiendo buenas prácticas de gestión de recursos. La función main presenta una estructura idéntica a BubbleArr, facilitando la comparación directa entre ambas estrategias. Esta variante es el punto de partida para comprender el patrón de ordenamiento indirecto que se repite en MergePtr y QuickPtr.

SortMerge: Ordenamiento por Mezcla

El módulo SortMerge contiene dos implementaciones del algoritmo Merge Sort, que sigue la estrategia de divide y venceras: divide el arreglo en dos mitades, ordena recursivamente cada mitad y las fusiona en un arreglo ordenado. Ambas variantes operan sobre los mismos tres tipos de datos y se distinguen por el uso de ordenamiento directo o indirecto.

MergeArr: Merge Sort Directo con Arreglos

El programa MergeArr implementa Merge Sort modificando directamente el arreglo de datos. La clase Ordenador expone los métodos mergeSortEnteros, mergeSortChars y

mergeSortPersonas, cada uno acompañado de su función auxiliar merge correspondiente. La función mergeSort recibe los índices izquierdo y derecho del subarreglo a ordenar, calcula el punto medio y se llama recursivamente sobre ambas mitades antes de invocar merge para fusionarlas en orden.

La función merge utiliza arreglos temporales para almacenar las dos mitades, las recorre comparando elemento a elemento y copia el menor de vuelta al arreglo original. Esto garantiza estabilidad en el ordenamiento (elementos iguales mantienen su orden relativo original) y una complejidad temporal O(n log n) en todos los casos, a diferencia de Bubble Sort. El costo adicional es el uso de memoria auxiliar O(n) durante la función.

Este programa es especialmente eficiente para ordenar arreglos grandes en comparación con BubbleArr, aunque su implementación es más compleja. La separación entre la lógica de ordenamiento (en la clase Ordenador) y la interacción con el usuario (en main) sigue el principio de responsabilidad única y facilita la reutilización del algoritmo en distintos contextos.

MergePtr: Merge Sort Indirecto con Arreglo de Índices

El programa MergePtr aplica la misma lógica recursiva de Merge Sort pero de forma indirecta. El arreglo de datos permanece sin modificaciones; todas las operaciones de comparación e intercambio se realizan sobre un arreglo de índices int* idx asignado dinámicamente. Al concluir, arr[idx[i]] devuelve el i-ésimo elemento en orden ascendente, preservando la posición original de cada dato en el arreglo principal.

La fusión en la variante indirecta requiere un arreglo temporal de índices en lugar de un arreglo temporal de datos, lo que reduce el costo de copia cuando los elementos son objetos de gran tamaño. En el caso de los objetos Persona, esto implica que en ningún momento se crea una copia temporal de los datos de nombre y edad; solo se manipulan enteros de cuatro bytes por elemento.

Al igual que BubblePtr, MergePtr libera la memoria de los arreglos de índices con delete[] al finalizar. La complejidad temporal se mantiene en O(n log n) y la espacial en O(n) para el arreglo de índices temporales. Este programa ilustra cómo el patrón de ordenamiento indirecto escala a algoritmos más eficientes que el Bubble Sort, manteniendo las mismas ventajas en el manejo de objetos complejos.

SortQuick: Ordenamiento Rapido

El módulo SortQuick contiene dos implementaciones del algoritmo Quick Sort. A diferencia de Merge Sort, Quick Sort realiza la division del problema mediante una funcion de particion que selecciona un pivote y reordena los elementos de modo que los menores queden a su izquierda y los mayores a su derecha, para luego ordenar recursivamente cada partición.

QuickArr: Quick Sort Directo con Arreglos

El programa QuickArr implementa Quick Sort con intercambio directo de elementos. La clase Ordenador expone las funciones partición Enteros, partición Chars y particion Personas, que seleccionan el último elemento del subarreglo como pivote, recorren el arreglo colocando los elementos menores al pivote a su izquierda y devuelve el índice final del pivote. La función quickSort invoca recursivamente a la partición sobre los dos subrangos resultantes.

La complejidad temporal promedio de Quick Sort es O(n log n), lo que lo hace comparable a Merge Sort en la práctica. Sin embargo, en el peor caso (arreglo ya ordenado con elección del último elemento como pivote) degenera a O(n^2). Su ventaja sobre Merge Sort es que opera in-place sin necesidad de arreglos auxiliares, por lo que su complejidad espacial es O(log n) para la pila de recursión en el caso promedio.

Este programa aplica la misma estructura de clase Persona con comparación por edad que los módulos anteriores, lo que facilita la comparación directa del rendimiento y la legibilidad del código entre los tres algoritmos de ordenamiento. La función main sigue el mismo patrón de entrada/salida que BubbleArr y MergeArr, permitiendo al estudiante enfocarse en las diferencias algorítmicas sin distracciones de interfaz.

QuickPtr: Quick Sort Indirecto con Arreglo de Índices

El programa QuickPtr implementa Quick Sort de forma indirecta mediante un arreglo de índices int* idx. La función de partición compara arr[idx[i]] con el pivote arr[idx[fin]] y reordena los índices en lugar de los datos. Esto garantiza que el arreglo original permanezca intacto y que el orden lógico se recupere accediendo a arr[idx[i]] al finalizar.

Al igual que QuickArr, esta variante opera in-place sobre el arreglo de índices, sin necesidad de arreglos auxiliares de datos durante la partición. Esto la hace eficiente tanto en tiempo como en espacio cuando los elementos son objetos grandes. La complejidad temporal y espacial es idéntica a QuickArr, pero con la ventaja adicional de que los intercambios son de enteros, no de objetos completos.

QuickPtr cierra el conjunto de seis programas de ordenamiento del proyecto, completando la matriz de tres algoritmos por dos estrategias de acceso. La memoria de los arreglos de índices se libera con delete[] al terminar. Este programa, junto con BubblePtr y MergePtr, demuestra que el patrón de ordenamiento indirecto es aplicable de manera uniforme a cualquier algoritmo de ordenamiento basado en comparaciones.

Pilas: Estructura de Datos LIFO

El módulo Pilas contiene seis implementaciones de una pila de enteros (Last-In, First-Out). Todas heredan de la clase abstracta PilaADT, que define la interfaz común: push, pop, top, size, isEmpty y mostrar. El programa principal de cada variante instancia la implementación concreta y la conecta al menú interactivo MenuPila, que permite al usuario apilar, desapilar, consultar el tope y visualizar el contenido de la pila.

Implementación con Arreglos Estáticos

Pilas/Arreglos/DatoBase. Esta implementación almacena los enteros directamente en un arreglo estático int datos[CAPACIDAD_MAX] con una capacidad máxima de 100 elementos. El método push incrementa el contador cantidad y deposita el valor en datos[cantidad - 1]; pop decrementa el contador sin borrar el dato; top devuelve datos[cantidad - 1]. Al no emplear memoria dinámica, no existe riesgo de fragmentación ni necesidad de destructor.

La estructura es la más sencilla de todas las variantes y sirve como punto de referencia conceptual. Su principal limitación es la capacidad fija: si el usuario intenta apilar más de 100 elementos, la operación debe rechazarse. En contrapartida, el acceso al tope es O(1) y no hay sobrecarga por asignación dinámica, lo que la hace adecuada para entornos con memoria restringida o cuando el tamaño máximo de la pila es conocido de antemano.

El menú interactivo MenuPila se reutiliza en las seis variantes sin modificación, lo que ilustra el principio de sustitución de Liskov: cualquier implementación concreta de PilaADT puede ser usada donde se espera un puntero a PilaADT. Esto permite cambiar la implementación subyacente sin alterar el código del menú, demostrando el valor del polimorfismo en el diseño orientado a objetos.

Pilas/Arreglos/NuevoDato. Esta variante es funcionalmente idéntica a Arreglos/DatoBase, pero los enteros se envuelven en una clase auxiliar NodoPila { int valor; } antes de almacenarse en el arreglo NodoPila nodos[CAPACIDAD_MAX]. La lógica de push, pop y top accede al campo valor del objeto correspondiente.

El propósito de esta variante es mostrar cómo escalar la estructura cuando el dato almacenado deja de ser un tipo primitivo y se convierte en un objeto más complejo. Si en el futuro se desea almacenar, por ejemplo, objetos con múltiples campos, bastaría con agregar los atributos necesarios a NodoPila sin cambiar la lógica de la pila. Esto separa la estructura de datos del tipo de dato, facilitando la extensión.

Desde el punto de vista de rendimiento, el impacto de envolver el entero en NodoPila es despreciable en este contexto, ya que el compilador puede optimizar el acceso a structs simples. Sin embargo, si NodoPila creciera para incluir cadenas u otros objetos pesados, la diferencia respecto a Punteros/NuevoDato (que usa lista enlazada) seria mas significativa en términos de localidad de caché.

Implementación con Punteros y Memoria Dinámica

Pilas/Punteros/DatoBase. Esta implementación utiliza un arreglo dinámico int* datos asignado con new. El constructor recibe una capacidad inicial (por defecto 4) y la almacena en el campo capacidad. Cuando push detecta que cantidad == capacidad, invoca redimensionar(), que duplica la capacidad creando un nuevo arreglo, copiando los datos existentes y liberando el anterior. El destructor llama a delete[].

La estrategia de duplicación garantiza que el costo amortizado de push sea O(1): aunque ocasionalmente se necesita una operación O(n) para redimensionar, esta ocurre cada vez que el tamaño se duplica, por lo que el costo total de n inserciones es O(n). Esta técnica es la misma que emplea std::vector en la STL y es un ejemplo clásico de análisis amortizado de algoritmos.

Esta variante elimina la limitación de capacidad fija de las implementaciones con arreglo estático, a costa de una mayor complejidad de implementación y el riesgo de errores de memoria si no se gestiona correctamente el ciclo de vida de los punteros. El programa ilustra las responsabilidades del programador en C++ respecto a la gestión manual de memoria, en contraste con la automatización que ofrece la STL.

Pilas/Punteros/NuevoDato. A diferencia de Punteros/DatoBase, esta variante implementa la pila como una lista enlazada simple. Cada elemento se almacena en un objeto NodoPila que contiene int valor y un puntero NodoPila* siguiente. El campo tope de la clase apunta al nodo en la cima de la pila.

push crea un nuevo NodoPila con new, establece su siguiente apuntando al tope actual y actualiza tope. pop guarda el puntero al nodo tope, actualiza tope al siguiente nodo y libera el nodo eliminado con delete. Esta estrategia garantiza que cada operación sea O(1) sin necesidad de redimensionamiento, ya que cada nodo se asigna individualmente.

La principal desventaja de la lista enlazada frente al arreglo dinámico es la menor localidad de caché: los nodos pueden estar dispersos en el heap, lo que dificulta el aprovechamiento de la línea de cache del procesador. Sin embargo, la lista enlazada no desperdicia memoria reservada pero no utilizada, a diferencia del arreglo dinámico que puede tener hasta la mitad de su capacidad libre tras un redimensionamiento.

Implementación con la Biblioteca Estándar STL

Pilas/Librerías/DatoBase. Esta implementación delega toda la lógica de almacenamiento en std::stack de la biblioteca estándar de C++. Los métodos de PilaADT simplemente invocan las operaciones equivalentes del contenedor STL: push llama a pila.push(), pop a pila.pop(), top a pila.top() y size a pila.size(). No es necesario gestionar la memoria manualmente.

std::stack es un adaptador de contenedor que por defecto utiliza std::deque como estructura subyacente, aunque puede configurarse para usar std::vector o std::list. Esto garantiza un comportamiento robusto, bien probado y eficiente, con complejidad O(1) para todas las operaciones de pila. El código resultante es el más conciso de las seis variantes, con mínimo riesgo de errores de implementación.

Esta variante ilustra el principio de no reinventar la rueda: cuando la STL ofrece exactamente lo que se necesita, usarla directamente produce código más legible, mantenible y confiable que cualquier implementación manual. El patrón de envolver la STL en una interfaz ADT propia es útil en contextos donde se desea poder sustituir la implementación subyacente sin afectar al código cliente.

Pilas/Librerías/NuevoDato. Esta ultima variante del módulo Pilas utiliza std::stack, almacenando objetos NodoPila { int valor; } en lugar de enteros directamente. Las operaciones de push convierten el entero recibido en un objeto NodoPila antes de insertarlo; top extrae el campo valor del objeto en la cima.

Esta variante demuestra que la STL puede almacenar objetos de tipo definido por el usuario de manera transparente, siempre que estos sean copiables o movibles. NodoPila es un objeto trivialmente copiable, por lo que std::stack lo maneja sin requerir implementación de constructores especiales.

Comparada con Librerías/DatoBase, esta implementación no ofrece ventajas funcionales o de rendimiento en este contexto específico, ya que NodoPila encapsula un único entero. Su valor radica en el ejercicio didáctico: mostrar que el patrón de dato envoltorio es compatible con contenedores STL, preparando al estudiante para escenarios donde NodoPila contenga múltiples campos o lógica adicional.

Colas: Estructura de Datos FIFO

El módulo Colas contiene seis implementaciones de una cola de enteros (First-In, First-Out). Todas se heredan de la clase abstracta ColaADT, que define: enqueue, dequeue, front, size, isEmpty y mostrar. El menú interactivo MenuCola conecta cualquier implementación concreta con el usuario, siguiendo el mismo patrón polimórfico que MenuPila.

Implementación con Arreglos Estáticos

Colas/Arreglos/DatoBase. La cola almacena enteros en int datos[CAPACIDAD_MAX] con dos índices: frente (inicio lógico de la cola) y cantidad (número de elementos presentes). enqueue deposita el nuevo valor en la posición (frente + cantidad) % CAPACIDAD_MAX, implementando un arreglo circular para reutilizar el espacio liberado por dequeue. dequeue avanza el índice frente en módulo CAPACIDAD_MAX.

El arreglo circular resuelve el problema de derroche de espacio que tendría una cola con arreglo lineal donde dequeue nunca lidera las posiciones del inicio. Todas las operaciones son O(1) y no se requiere memoria dinámica. La capacidad máxima se fija en 100 elementos, lo que limita su uso a escenarios donde el tamaño de la cola sea acotado y conocido.

Esta implementación es la mas eficiente en términos de localidad de caché de las seis variantes, ya que todos los datos residen en un bloque contiguo de memoria estática. Al no requerir llamadas al sistema para gestión de memoria, es adecuada para sistemas embebidos o entornos de tiempo real donde la predecibilidad de la latencia es critica.

Colas/Arreglos/NuevoDato. Esta variante encapsula el almacenamiento en una clase auxiliar Datos, que contiene int arr[CAPACIDAD_MAX], frente y cantidad, junto con los métodos de conveniencia tamano() y estaVacia(). La clase ColaEnteros opera exclusivamente a través de su campo Datos d, delegando la responsabilidad de almacenamiento a dicho objeto.

La separación entre la lógica de cola (ColaEnteros) y la lógica de almacenamiento (Datos) aplica el principio de composición sobre herencia. Si en el futuro se desea cambiar la estructura interna del almacenamiento, basta con modificar la clase Datos sin tocar ColaEnteros. Esta separacion de responsabilidades facilita las pruebas unitarias de cada componente de forma independiente.

Aunque el encapsulamiento adicional no altera la complejidad algoritmica ni el rendimiento en tiempo de ejecución, si introduce un nivel de indirection adicional que puede dificultar la lectura del código para estudiantes principiantes. Su inclusión en el proyecto busca exponer este patrón de diseño y contrastar su legibilidad frente a la variante DatoBase.

Implementación con Punteros y Memoria Dinámica

Colas/Punteros/DatoBase. Esta implementacion usa int* arr en el heap con capacidad inicial parametrizable. El campo final rastrea la posición del último elemento insertado. Cuando enqueue detecta que final + 1 == capacidad, invoca redimensionar(), que crea un nuevo arreglo con el doble de capacidad, copia los elementos y libera el anterior. El destructor llama a delete[].

A diferencia de la implementacion con arreglo circular estático, esta variante no implementa lógica circular: los elementos se depositan secuencialmente y dequeue desplaza todos los elementos una posición hacia el inicio. Esto implica que dequeue tiene complejidad O(n), lo que la hace menos eficiente que la variante circular para operaciones frecuentes de desencolado.

La ventaja de esta implementación es la capacidad ilimitada: la cola puede crecer tanto como lo permita la memoria disponible. Es apropiada para escenarios donde el volumen de datos es impredecible y la eficiencia de dequeue no es crítica. La lógica de redimensionamiento es directamente comparable a la de Pilas/Punteros/DatoBase, reforzando el patrón de arreglo dinámico con crecimiento amortizado.

Colas/Punteros/NuevoDato. Combina el almacenamiento dinámico con la clase auxiliar Datos, que encapsula int* arr, frente, final y capacidad, incluido su propio constructor y destructor. ColaEnteros opera a través del objeto Datos d, invocando redimensionar() cuando se agota la capacidad. El destructor de ColaEnteros no necesita liberar memoria directamente, pues el destructor de Datos se encarga de ello.

Esta variante ilustra el principio RAII (Resource Acquisition Is Initialization): la clase Datos adquiere recursos en su constructor y los libera en su destructor, garantizando que no haya fugas de memoria incluso si se lanza una excepción. Este patrón es fundamental en C++ moderno y anticipa el uso de smart pointers como std::unique_ptr.

La doble encapsulación (ColaEnteros contiene Datos que contiene el arreglo) aumenta la cohesión y reduce el acoplamiento entre los componentes del programa. Sin embargo, complica ligeramente la trazabilidad del flujo de control para quien lee el código por primera vez, por lo que se recomienda acompañar su estudio con diagramas de clases que muestren la relación de composición.

Implementación con la Biblioteca Estándar STL

Colas/Librerias/DatoBase. Esta implementación encapsula std::queue. Los métodos de ColaADT invocan directamente cola.push() para enqueue, cola.pop() para dequeue (que elimina el elemento al frente), cola.front() para front y cola.size() para size. La STL gestiona internamente la memoria y la estructura de datos.

std::queue es un adaptador que por defecto usa std::deque como contenedor subyacente, lo que garantiza complejidad O(1) amortizada para enqueue y O(1) para dequeue, sin el problema de desplazamiento de elementos que tenía Punteros/DatoBase. Esta es la implementación mas eficiente y robusta de las seis variantes del módulo Colas.

Al igual que en el módulo Pilas, la variante STL demuestra que el código de negocio (la interfaz ColaADT y el menú Menu MenuCola) puede permanecer completamente ignorante de los detalles de implementación subyacentes. Esto refuerza el valor de las interfaces abstractas y el polimorfismo como mecanismos para desacoplar la interfaz de la implementación.

Colas/Librerías/NuevoDato. Utiliza std::queue, donde NodoEntero { int valor; } envuelve el entero. enqueue convierte el entero recibido en un objeto NodoEntero y lo inserta en la cola; front extrae el campo valor del objeto al frente; dequeue elimina el objeto del frente mediante cola.pop().

Como en la variante equivalente de Pilas, el uso de NodoEntero con std::queue es funcionalmente equivalente al uso de int, pero prepara la estructura para escenarios donde el objeto almacenado sea más complejo. La STL maneja la construcción, copia y destrucción de NodoEntero de forma automática, siempre que el objeto satisfaga los requisitos de CopyConstructible.

Esta variante cierra el módulo Colas completando la matriz de seis implementaciones. Junto con las otras cinco variantes, permite al estudiante visualizar de forma comparativa como una misma abstracción (la cola FIFO) puede realizarse con diferentes compromisos entre simplicidad, eficiencia, uso de memoria y extensibilidad del código.

Listas: Estructura de Datos Lineal

El módulo Listas contiene seis implementaciones de una lista de enteros. Todas heredan de

ListaADT, que define: insertAt (insertar al final), removeAt (eliminar el primer elemento), get (consultar el primer elemento), size, isEmpty y mostrar. El menú interactivo MenuLista conecta las implementaciones con el usuario.

Implementación con Arreglos Estáticos

Listas/Arreglos/DatoBase. La lista almacena enteros en int datos[CAPACIDAD_MAX] con un contador cantidad. insertAt agrega el nuevo valor en datos[cantidad] e incrementa el contador; removeAt desplaza todos los elementos una posición hacia la izquierda para eliminar el primero y decrementa cantidad; get devuelve datos[0].

La operación removeAt tiene complejidad O(n) debido al desplazamiento de elementos, lo que la hace ineficiente para listas con frecuentes eliminaciones al inicio. insertAt al final es O(1) siempre que no se exceda la capacidad. Esta implementación es didácticamente valiosa por su simplicidad y por ilustrar claramente el costo del desplazamiento en arreglos.

Esta variante es la referencia base del módulo Listas, equivalente a lo que BubbleArr es en el módulo SortBubble. Sirve para contrastar con las implementaciones de lista enlazada (Punteros/NuevoDato) donde removeAt puede realizarse en O(1) sin desplazamiento. La elección entre arreglo y lista enlazada depende del patrón de uso predominante en la aplicación.

Listas/Arreglos/NuevoDato. Idéntica lógica a Arreglos/DatoBase pero los valores se almacenan como NodoEntero nodos[CAPACIDAD_MAX], donde NodoEntero { int valor; } envuelve el entero. Las operaciones acceden al campo valor del nodo correspondiente.

La clase NodoEntero cumple el mismo rol que NodoPila en Pilas y NodoEntero en Colas: abstraer el tipo de dato almacenado para facilitar la extensión futura. Si se desea una lista de objetos con múltiples campos, basta con agregar atributos a NodoEntero y actualizar las comparaciones pertinentes, sin necesidad de rediseñar la estructura de la lista.

Desde el punto de vista del rendimiento, el acceso a nodos[i].valor tiene exactamente el mismo costo que acceder a datos[i] en la variante DatoBase, ya que NodoEntero es un struct de un solo campo sin funciones virtuales. El compilador optimiza el acceso al miembro, generando código maquina idéntico en ambos casos para arquitecturas modernas.

Implementación con Punteros y Memoria Dinámica

Listas/Punteros/DatoBase. Esta variante declara int datos[CAPACIDAD_MAX] de manera similar a Arreglos/DatoBase, pero su arquitectura está diseñada como paso intermedio hacia la implementación con punteros reales. La interfaz y el comportamiento son equivalentes a la versión de arreglo estático, y el archivo de implementación puede extenderse para reemplazar el arreglo por un puntero dinámico.

Su inclusión en la categoría Punteros dentro de la jerarquia del repositorio sugiere que representa el patrón base sobre el cual se construye NuevoDato, siguiendo la misma lógica de progresión que se observa en los módulos Pilas y Colas. La estructura de archivos (pila.h / lista.h separados de main.cpp) facilita sustituir la implementación sin modificar el programa principal.

Esta variante es útil como ejercicio de refactorización: el estudiante puede tomar el codigo de Arreglos/DatoBase y transformarlo progresivamente, primero introduciendo un puntero dinámico y luego evolucionando hacia la lista enlazada de Punteros/NuevoDato. Esta progresión gradual ayuda a comprender las implicaciones de cada cambio de diseño.

Listas/Punteros/NuevoDato. Esta es la única implementacion del módulo Listas que usa una lista enlazada simple. Cada nodo es un objeto Nodo con int dato y Nodo* siguiente. La clase ListaEnteros mantiene punteros cabeza y cola: insertAt crea un nodo con new, lo enlaza al final mediante cola->siguiente y actualiza cola; removeAt desenlaza el nodo cabeza y lo elimina con delete.

Gracias al puntero cola, insertAt es O(1) sin necesidad de recorrer la lista. removeAt también es O(1) ya que elimina directamente el nodo apuntado por cabeza. Esta combinación hace de Punteros/NuevoDato la implementación más eficiente del módulo para cargas de trabajo con frecuentes inserciones al final y eliminaciones al inicio, que es precisamente el patrón de acceso de una cola o buffer FIFO.

El destructor recorre la lista desde cabeza hasta nullptr, liberando cada nodo con delete. Si no se implementara, cada nodo quedaría sin liberar al destruir el objeto ListaEnteros, produciendo una fuga de memoria proporcional al número de elementos. Esta variante es el ejemplo más completo del módulo en cuanto a gestión correcta del ciclo de vida de los recursos.

Implementación con la Biblioteca Estándar STL

Listas/Librerias/DatoBase. Encapsula std::list. insertAt invoca lista.push_back(), agregando al final en O(1); removeAt invoca lista.pop_front(), eliminando el primer elemento en O(1); get devuelve lista.front(). std::list es una lista doblemente enlazada que gestiona automáticamente la memoria de sus nodos.

A diferencia de std::vector, std::list garantiza O(1) para inserciones y eliminaciones en cualquier posición conocida (iterador), lo que la hace ideal para las operaciones definidas en ListaADT. El costo es una mayor sobrecarga de memoria por nodo (cada nodo almacena dos punteros adicionales para la lista doblemente enlazada) y peor localidad de cache que std::vector.

Esta variante completa el patrón de las tres implementaciones STL del proyecto (std::stack, std::queue, std::list), mostrando que la STL cubre las tres estructuras de datos lineales clásicas con contenedores o adaptadores especializados. La consistencia en el diseño de la STL facilita la transición entre estas estructuras sin necesidad de aprender paradigmas completamente distintos.

Listas/Librerias/NuevoDato. Utiliza std::list, donde NodoEntero { int valor; } envuelve el entero. Las operaciones se convierten entre int y NodoEntero según sea necesario: insertAt construye un NodoEntero y lo inserta con push_back; get extrae el campo valor del objeto al frente.

Esta es la última de las 24 implementaciones del proyecto y cierra la sistemática matriz de seis variantes por módulo. Su diseño es coherente con todas las variantes NuevoDato anteriores, reforzando el patrón de objeto envoltorio como mecanismo de extensión. La STL maneja la memoria de los objetos NodoEntero de forma automática, igualando en comodidad a las variantes DatoBase.

La inclusión de esta variante en el proyecto completa la demostración de que el patrón Nuevo Dato es ortogonal al mecanismo de almacenamiento: puede aplicarse tanto con arreglos estáticos como con memoria dinámica o con contenedores STL. Esta ortogonalidad es una propiedad deseable en el diseño de software, ya que permite combinar decisiones de diseño de forma independiente sin generar incompatibilidades.